

How the cloning works, and why this data

FDA behavior-clone system (“parallel”) — deep dive for interview prep. All code below is the exact production code.

1. The three-player game

The regulatory world is a three-player game, so the system builds three clones, one per player. Any FDA change moves all three; predicting only one leaves a blind spot.

- **Enforcement clone** — models ORA/CDER compliance behavior: what gets cited, how fast things escalate, what triggers import alerts or injunctions.
- **Reviewer clone** — models premarket reviewer behavior: what questions get asked, what becomes refuse-to-accept criteria, how predicates get challenged.
- **Sponsor clone** — models industry behavior: how companies adapt submissions, run recalls, and lobby through dockets.

2. Why these data sources

Each source was chosen because it records **behavior**, not just rules:

- **Warning letters** are the richest public record of how FDA enforcement *behaves*. They reveal the actual escalation ladder (483 → warning letter → import alert / seizure / injunction), which CFR parts get cited together, and aggravating factors — e.g. 501(j) inspection refusal — that accelerate cases.
- **510(k) / De Novo / PMA decision summaries + guidance documents** show what reviewers actually asked for and accepted. That observed questioning behavior is exactly what the reviewer clone must imitate.
- **Recall records** are cross-stakeholder evidence: a single record shows enforcement classification behavior *and* sponsor field-action behavior together.
- **Federal Register notices** capture policy shifts that change reviewer and enforcement posture *before* they land in guidance.

The demo seed uses real titles, dates, and URLs from these sources; the analyses and predictions built on them are synthetic and labeled as demo data.

3. What the advanced version adds

The first clones were just system prompts plus the Chat API. The advanced version gives each clone an **evidence base**:

- **Pattern library** (`src/clones/patterns.py`) — 9 behavior patterns, each with trigger phrases, typical actions, evidence citations, confidence, and supporting-record counts. Visible on the Clones page.
- **Deterministic matcher** — for each new change, every pattern is scored by *trigger coverage* × *confidence*, per stakeholder, with zero API calls.
- **Pattern-grounded predictions** — matched patterns are injected into the clone's prompt as its evidence base, and each prediction reports exactly which patterns fired, with scores.

- **Mining loop closed** — `mine_and_store()` persists Task-mined patterns into the library, so future clones reason from them.

4. The exact algorithm

4.1 Pattern matching — `src/clones/patterns.py`

Build one lowercase haystack from the analysis (summary + change type + product codes + submission types). For each pattern belonging to this stakeholder, count how many trigger phrases appear; $\text{score} = (\text{matched} / \text{total triggers}) \times \text{pattern confidence}$. Sort descending, keep the top 3. Deterministic, no LLM call, and every fired pattern carries its score, confidence, and the exact triggers that matched — which is what makes predictions explainable.

```
def match_patterns(analysis, patterns, stakeholder, top_k=3):
    text = " ".join([
        analysis["summary"],
        analysis["change_type"],
        " ".join(analysis["affected_product_codes"]),
        " ".join(analysis["affected_submission_types"]),
    ]).lower()
    scored = []
    for p in patterns:
        if p.stakeholder != stakeholder or not p.triggers:
            continue
        matched = [t for t in p.triggers if t.lower() in text]
        if not matched:
            continue
        score = (len(matched) / len(p.triggers)) * p.confidence
        scored.append((p, round(score, 3), matched))
    scored.sort(key=lambda s: s[1], reverse=True)
    return scored[:top_k]
```

Listing 1 — `match_patterns`: the retrieval algorithm.

4.2 Clone prediction — `src/clones/clone.py`

The matched patterns become an evidence block inside the prompt. The model must ground its prediction in the patterns that apply and name them. The result echoes `patterns_used`, so every prediction is traceable to evidence:

```
def respond_to_change(self, change_analysis):
    pattern_lines = []
    for pattern, score, matched in self.patterns:
        ev = "; ".join(pattern.evidence_citations[:2])
        pattern_lines.append(
            f"- {pattern.name} (relevance {score:.2f}, confidence "
            f"{pattern.confidence:.0%}, fired on: {' '.join(matched)}): "
            f"{pattern.description} Evidence: {ev}"
        )
    pattern_block = ""
    if pattern_lines:
        pattern_block = (
            "\n\nKnown behavior patterns for this stakeholder "
            "(distilled from FDA records). Ground your prediction "
            "in the ones that apply and name them:\n"
            + "\n".join(pattern_lines)
        )
    question = (
        "A new FDA regulatory change has been analyzed "
        "(details in Context). "
```

```

        f"As a {self.profile['role']], predict how this stakeholder "
        "behaves in response: concrete actions, likely timelines, "
        "and what would change your prediction." + pattern_block +
        " End with a confidence level (high/medium/low) and why."
    )
    context = json.dumps(change_analysis, indent=2)[:12000]
    out = self.ask(question, context=context)
    return {
        "clone": self.profile["name"],
        "role": self.profile["role"],
        "prediction": out["answer"],
        "citations": out["citations"],
        "patterns_used": [
            {"name": p.name, "score": score,
             "confidence": p.confidence,
             "matched_triggers": matched}
            for p, score, matched in self.patterns
        ],
    }
}

```

Listing 2 — respond_to_change: evidence-grounded prediction.

4.3 Pattern mining — src/clones/mining.py

FindAll assembles the corpus of FDA records per stakeholder; a Task run distills recurring behavior patterns under a JSON-schema contract (BEHAVIOR_PATTERN_SCHEMA), so the output is typed and parseable:

```

def mine_patterns(client, stakeholder, seed_query=None,
                  max_entities=25):
    query = seed_query or SEED_QUERIES[stakeholder]
    found = client.findall(
        objective=f"Find FDA records showing {stakeholder} "
        f"behavior patterns: {query}",
        criteria={"stakeholder": stakeholder, "topic": query,
                 "max_entities": max_entities},
    )
    entities = found.get("entities", found.get("results", [])) or []
    run = client.task_run(
        objective=(
            f"Distill recurring {stakeholder} behavior patterns "
            "from these FDA records. Each pattern needs a name, "
            "the trigger conditions, and evidence citations.\n\n"
            f"Records:\n{json.dumps(entities)[:15000]}"),
        output_schema={
            "type": "object",
            "required": ["patterns"],
            "properties": {
                "patterns": {"type": "array",
                             "items": BEHAVIOR_PATTERN_SCHEMA},
            },
        },
    )
    run_id = run.get("run_id", run.get("id"))
    output = client.wait_for_task(run_id)
    if isinstance(output, dict):
        return output.get("patterns", [])
    return []

```

Listing 3 — mine_patterns: FindAll corpus, Task distillation.

4.4 Fan-out simulation — src/simulate/runner.py

Every clone runs independently against the same analysis. One clone's failure is captured as an error payload instead of aborting the batch — partial results beat no results:

```
def run_simulation(client, change_analysis, clones=None):
    clones = clones if clones is not None else build_clones(client)
    predictions = {}
    for clone in clones:
        name = clone.profile["name"]
        try:
            predictions[name] = clone.respond_to_change(change_analysis)
        except Exception as exc: # one clone failing must not kill the batch
            predictions[name] = {"clone": name, "error": str(exc)}
    return {
        "change_summary": change_analysis.get("summary", ""),
        "change_type": change_analysis.get("change_type", ""),
        "severity": change_analysis.get("severity"),
        "predictions": predictions,
    }
```

Listing 4 — run_simulation: fault-tolerant fan-out.

5. Worked example

Analysis: *“FDA issued a warning letter to Jabil Inc., a large CDMO, citing aseptic processing violations.”* (change_type: warning letter, product codes [LZG], submission types [510(k)]). Scoring the three enforcement patterns:

- inspection-refusal-escalation: no triggers match → does not fire.
- cdm0-warning-letter-cluster: “cdmo”, “jabil” match → coverage $2/4 = 0.50 \times \text{confidence } 0.75 = \mathbf{0.375}$ → fires.
- glp1-compounding-crackdown: no triggers match → does not fire.

The enforcement clone therefore reasons from cdm0-warning-letter-cluster (relevance 0.375, confidence 75%) and cites it in its prediction. This is the actual output of the algorithm, verified in a test run.

6. Architecture recap

Sense (Search / Extract / Monitor) → Understand (Task + schema contracts) → Clone (FindAll + Task → pattern library; Chat + matcher → predictions) → Simulate (fan-out) → Surface (FastAPI dashboard).

The key separation: **mining** (batch, expensive, LLM) vs. **matching** (deterministic, free, explainable) vs. **reasoning** (LLM, grounded in matched evidence). That is what makes the clones more than personas in a prompt.